# Ecosphere Technical DOC

## 1. Overview

### What Ecosphere Is

Ecosphere is a sustainability intelligence platform that combines two product threads into one application:

- `Ecosphere`: carbon intelligence, emissions visibility, and sustainability analytics
- `POWAMOV`: kinetic road energy harvesting infrastructure monitoring and simulation

The current application presents both as a single web experience with shared navigation, a common visual system, and a frontend/backend workspace structure intended to grow into a more integrated operational platform.

### Primary Purpose of the Platform

The platform's current purpose is to:

- visualize POWAMOV infrastructure performance and operational status
- demonstrate a future digital twin and predictive maintenance workflow
- show carbon and energy intelligence using regional datasets
- support partner conversations, technical demos, and product validation

### Current Version State

The codebase is in an advanced prototype / partner-demo state rather than a production deployment state.

There is also a naming/version mismatch in the repository that is worth calling out explicitly:

- the monorepo root package is currently named `ecospher v2.1`
- product copy in the workspace still refers to `Ecosphere 2.0`
- the frontend package version is still `0.0.0`

Practically, this means the platform has moved beyond a throwaway prototype, but it has not yet been normalized into a formal release process or a production-ready versioning model.

### Core Capabilities Today

- POWAMOV Command Center with live-style telemetry and node health monitoring
- Digital Twin simulations for road strip compression and corridor behavior
- Predictive Maintenance dashboards with alert and forecast views

- Energy Analytics tied to harvested energy and carbon offset storytelling
- Regional Carbon Analytics using bundled Botswana and South Africa datasets
- Enterprise Carbon Calculator for Scope 1, 2, and 3 scenario modeling
- Emissions Tracker with simulated live data and Botswana regional energy context
- Operations Hub for deployment, team, and resource storytelling
- Offline account/profile/settings flows for local demo usage

# 2. Application Architecture

## Monorepo Structure

The application is a `pnpm` workspace monorepo with separate artifacts for the frontend and backend, plus shared packages for API contracts and generated clients.

| Area | Path | Role |
|---|---|---|
| Frontend app | `artifacts/powamov` | React/Vite product UI |
| Backend app | `artifacts/api-server` | Express API serving mock POWAMOV data |
| API spec | `lib/api-spec` | OpenAPI contract for backend routes |
| Generated client | `lib/api-client-react` | Orval-generated React Query hooks and fetch wrapper |
| Shared validation | `lib/api-zod` | Zod schemas used by the API layer |
| Database package | `lib/db` | Drizzle/Postgres scaffold for future persistence |
| Dev orchestration | `scripts` | Concurrent frontend/backend local startup |

## Frontend Architecture

The frontend is a React 19 + Vite single-page application. It uses:

- `Wouter` for client-side routing
- `TanStack Query` for API data fetching and caching
- `Zustand` for a small amount of client state
- local component state for page-level simulation and interaction logic

- `localStorage` for auth/session and some user-created scenarios

The frontend is organized around route-level pages in `src/pages`, wrapped by a shell layout with a sidebar and topbar in `src/components/layout`.

## Component Structure

At a high level, the frontend follows this structure:

- `src/main.tsx`
  - bootstraps the app
  - configures the generated API client base URL from `VITE_API_URL` when provided
- `src/App.tsx`
  - defines the route map
  - wraps the app in `QueryClientProvider`, theme provider, tooltip provider, and auth guard logic
- `src/components/layout/*`
  - shared shell, sidebar, topbar, and navigation framing
- `src/pages/*`
  - feature-level screens for POWAMOV, carbon intelligence, and operations
- `src/lib/demo-api.ts`
  - fallback demo datasets used when the API is unavailable or response shapes are invalid
- `src/stores/emissionsStore.ts`
  - local store for some emissions-related scenarios and persisted client data
- `src/utils/auth.ts`
  - offline local auth/session implementation using `localStorage`

## Routing Structure

The frontend routes are currently:

| Route | Screen | Access |
|---|---|---|
| `/` | Intelligence Overview dashboard | Protected |
| `/command` | POWAMOV Command Center | Protected |
| `/digital-twin` | Digital Twin | Protected |
| `/maintenance` | Predictive Maintenance | Protected |
| `/analytics` | Energy Analytics | Protected |
| `/carbon-analytics` | Regional Carbon Analytics | Protected |
| `/calculator` | Enterprise Carbon Calculator | Protected |
| `/tracker` | Emissions Tracker | Protected |
| `/collaborators` | Operations Hub | Protected |
| `/settings` | Settings | Protected |
| `/profile` | Profile | Protected |
| `/login` | Login / sign-up | Public |

All protected routes are gated by a frontend-only auth guard backed by `localStorage`. This is suitable for demos and local testing, but it is not a production authentication model.

### State Management

The application currently uses four state layers:

| State Type | Mechanism | Current Use |
|---|---|---|
| Server state | TanStack Query | API data such as telemetry, nodes, maintenance, analytics |
| Local app state | Zustand | Emissions scenarios and tracker entries |
| Page simulation state | React state/hooks | Digital Twin and tracker live simulation behavior |
| Persisted local user state | `localStorage` | Auth session, user profiles, saved calculator scenarios |

There is no centralized global domain state beyond these pieces. Most modules are intentionally page-scoped.

### Data Flow

The primary data flow in local development is:

1. React page calls a generated hook from `@workspace/api-client-react`
2. The generated client requests a relative `/api/*` route
3. Vite proxies `/api` to the local Express server during development
4. The Express backend returns mock or simulated POWAMOV data
5. The frontend renders that data and, in some pages, falls back to `demo-api.ts` if the API is unavailable

There are also modules that bypass the API entirely:

- Regional Carbon Analytics reads bundled JSON datasets directly
- Carbon Calculator is fully client-side
- Emissions Tracker mixes bundled JSON data with client-generated live simulation
- Operations Hub is built from static in-file objects
- Auth is fully local and does not call the backend

## 3. Core Modules

### Module Summary

| Module | Purpose | Data Sources | Current Functionality | Maturity |
|---|---|---|---|---|
| Command Center | Monitor POWAMOV node network performance | Mock API telemetry, mock node inventory, frontend demo fallback | Live-style KPI cards, node map, telemetry charts, node status grid | Strong demo / mock-backed MVP |
| Digital Twin | Simulate strip compression and corridor operations | Pure client-side simulation logic | Vehicle/strip animation, corridor health simulation, energy trends | Advanced simulation prototype |
| Predictive Maintenance | Show maintenance risk and alerting concepts | Mock API forecasts/alerts, node list, demo fallback | Forecast tables, alert views, health/degradation visualizations | Mock-backed prototype |
| Energy Analytics | Summarize harvested energy and impact | Mock API summary/history, demo fallback | KPI cards, historical trends, carbon-offset storytelling | Strong demo / mock-backed MVP |
| Regional Carbon | Compare regional grid carbon trends | Bundled BW/ZA JSON datasets | Monthly comparisons, renewable share and carbon-free energy views | Data-backed MVP module |
| Carbon Calculator | Estimate enterprise emissions | Client form inputs, hardcoded factor model, `localStorage` | Scope 1/2/3 calculation, saved scenarios, offset framing | Solid product prototype |
| Emissions Tracker | Show live-style emissions and regional context | `B_E_D.json`, client-side interval simulation | Streaming chart, regional intensity and RE views, region filtering | Demo-ready hybrid module |

| Module | Purpose | Data Sources | Current Functionality | Maturity |
|---|---|---|---|---|
| Operations Hub | Present deployments, teams, resources, and projects | Static hardcoded arrays | Operations storytelling UI with tabs and summaries | Concept/demo UI |

## POWAMOV Infrastructure

### Command Center

Purpose:

- act as the operational landing page for POWAMOV infrastructure
- show node network health, energy generation, and traffic activity

Data sources:

- `GET /api/telemetry/live`
- `GET /api/nodes`
- `GET /api/telemetry/history?hours=24`
- frontend fallback data from `src/lib/demo-api.ts`

Current functionality:

- current output in kW
- rolling 24-hour energy harvest
- total vehicle passes today
- node online/warning/offline counts
- San Francisco node map using mock coordinates
- 24-hour telemetry chart
- node-by-node status grid with degradation and total energy

Maturity:

- visually strong and technically structured for future real telemetry
- currently depends on mock backend data rather than live field integration

### Digital Twin

Purpose:

- demonstrate how POWAMOV infrastructure behavior could be simulated at both node and corridor level

Data sources:

- no live backend dependency
- entirely client-side simulated state

Current functionality:

- single-node six-strip vehicle compression simulation
- adjustable vehicle weight and derived vehicle classification
- animated strip activation and instantaneous force/energy outputs
- corridor-level city simulation for selected Gaborone arterials
- maintenance-style alert storytelling and corridor health summaries

Maturity:

- strong product demo and concept validation tool
- not yet connected to real sensor models, hardware telemetry, or a persisted simulation engine

#### Predictive Maintenance

Purpose:

- visualize component degradation and demonstrate a future proactive maintenance workflow

Data sources:

- `GET /api/maintenance/forecasts`
- `GET /api/maintenance/alerts`
- `GET /api/nodes`
- frontend fallback demo data

Current functionality:

- maintenance forecasts per node
- active alerts
- degradation and efficiency trend displays
- node health context for prioritization

Maturity:

- operationally useful as a demo layer
- the maintenance logic is still mock/generated, not model-driven or sensor-driven

## Carbon Intelligence

### Energy Analytics

Purpose:

- convert POWAMOV activity into sustainability and impact metrics

Data sources:

- `GET /api/analytics/summary`
- `GET /api/analytics/energy-history`
- frontend fallback demo data

Current functionality:

- total harvested energy
- carbon offset equivalents
- grid displacement indicators
- trend visualization over time

Maturity:

- strong storytelling module
- current outputs are derived from mock telemetry history rather than verified operational data

### Regional Carbon

Purpose:

- compare carbon intensity and renewable performance across Botswana and South Africa datasets

Data sources:

- `BW_2023_monthly.json`
- `BW_2024_monthly.json`
- `ZA_2023_monthly.json`
- `ZA_2024_monthly.json`

Current functionality:

- monthly comparison charts
- average intensity summaries
- renewable percentage and carbon-free energy views
- narrative regional insight framing

Maturity:

- one of the more data-grounded modules in the platform

- still based on bundled snapshot files rather than a live external data feed

### Carbon Calculator

Purpose:

- let organizations estimate Scope 1, 2, and 3 emissions using a guided input flow

Data sources:

- direct user input
- hardcoded emissions factors in the frontend
- scenario persistence in `localStorage`

Current functionality:

- enterprise emissions calculator across multiple categories
- region-aware electricity factor handling
- saved scenario support
- POWAMOV offset potential framing

Maturity:

- useful product prototype
- not yet backed by versioned factors, standards governance, or auditability controls

### Emissions Tracker

Purpose:

- present a real-time style emissions screen for live monitoring and regional analysis

Data sources:

- `B_E_D.json` Botswana regional energy dataset
- client-side interval simulation for the “live” stream

Current functionality:

- simulated live emissions chart
- selectable update frequency
- regional carbon intensity comparison
- renewable percentage analysis by region
- region filtering and alternative chart modes

Maturity:

- demo-ready hybrid module

- the real-time layer is synthetic; only the regional reference dataset is grounded in actual bundled data

### Operations

#### Operations Hub

Purpose:

- communicate deployment footprint, teams, resources, and related sustainability projects

Data sources:

- hardcoded arrays within the page component

Current functionality:

- deployment summaries
- team listings
- resource views
- environmental project summaries

Maturity:

- currently a narrative/operational demo surface
- not connected to live project systems, staffing tools, or asset tracking

## 4. Emissions Tracker Deep Analysis

### How It Currently Works

The Emissions Tracker is not connected to a live emissions API. Instead, it generates a synthetic live feed in the browser using `setInterval`. A `lastEmission` reference is randomly drifted over time, constrained within a fixed range, and appended to a rolling list of up to 30 chart points.

This creates a convincing live-monitoring experience for demos without requiring a backend stream.

### Data Sources Used

The tracker has two different data layers:

- simulated live feed:
  - generated in the browser
  - update intervals selectable by the user: `1s`, `5s`, `30s`, `1m`
- regional baseline dataset:

- sourced from `src/data/B_E_D.json`
- used to compute average carbon intensity and average renewable percentage per Botswana region

### Real-Time Architecture

The “real-time” architecture is entirely frontend-driven:

1. a timer fires at the selected interval
2. a new emission value is created by applying a random drift to the last point
3. the page updates local React state
4. Recharts re-renders the line chart from the rolling in-memory array

There is no WebSocket, SSE, queue, broker, stream processor, or backend push mechanism in the current implementation.

### Visualization Logic

The page renders three visualization modes:

- live line chart for simulated emissions over time
- bar or pie view for regional carbon intensity
- bar chart for regional renewable percentages

The UI also computes session-level summary cards such as:

- current live emission value
- short-term trend
- session average
- sample count

### Simulation vs Real Data

The tracker is a hybrid module:

- real:
  - the bundled regional dataset and its aggregated calculations
- simulated:
  - all “live” emissions points
  - session trend movement
  - streaming behavior

This means the module is strong for demos and partner discussion, but it should not be interpreted as a live operational emissions pipeline today.

# 5. Digital Twin Analysis

## What Is Currently Simulated

The Digital Twin page contains two simulation layers:

- node-level strip simulation
- city/corridor-level arterial simulation

At node level, the app simulates:

- six POWAMOV strips
- vehicle movement across the strips
- compression percentage per strip
- force in kN
- energy generation in Wh/mWh
- vehicle classes derived from selected weight
- rolling energy history and pass counts

At corridor level, the app simulates:

- traffic on four Gaborone corridors
- vehicles per minute
- average speed
- energy output
- heavy-vehicle share
- corridor health state
- maintenance-style alerts

## Node Structure

The node simulation uses a conceptual six-strip model:

- six strip centers are defined in the SVG scene
- a moving vehicle crosses them from left to right
- each strip computes compression based on the vehicle overlap and selected vehicle weight
- the page derives force and energy from that compression

This is currently a visual/interaction model, not a hardware-calibrated engineering model.

## Data Flow

The Digital Twin flow is fully local:

1. the user chooses weight and node context

2. an animation loop updates vehicle position
3. strip overlap is recalculated frame by frame
4. force, compression, and energy values are recomputed
5. historical arrays are updated in React state
6. charts and summary cards re-render

No API call is required for the current experience.

### UI Rendering Logic

The page uses:

- animated SVG rendering for the road and strip interaction
- Recharts for historical trends
- Framer Motion for transitions and interaction polish
- React tabbed views to switch between node simulation and city-level simulation

The result is a sophisticated UI simulation layer that is well suited for concept communication and product demo workflows.

## 6. Command Center Analysis

### Metrics Displayed

The Command Center currently displays:

- current power output
- today’s energy generated
- today’s vehicle passes
- nodes online / warning / offline
- infrastructure node map
- recent telemetry graph
- node-level degradation and total energy values

### Aggregation Logic

The main telemetry chart is built from the backend’s telemetry history response:

- the frontend requests `24` hours of history
- it takes the last 24 records
- it maps each record into chart-friendly values:
  - `time`

- `power`
- `energy`
- `passes`

The cards use the current live telemetry payload for totals and status counts.

### Data Sources

The page reads from:

- `GET /api/telemetry/live`
- `GET /api/nodes`
- `GET /api/telemetry/history`

When these responses are missing or invalid, the page falls back to demo data bundled in `src/lib/demo-api.ts`.

### What This Means Today

Architecturally, the Command Center is already shaped like a real monitoring UI. The missing piece is not the frontend pattern; it is the upstream data source. Replacing the mock telemetry provider with a real ingestion pipeline should fit the current route/hook/UI structure with limited frontend redesign.

## 7. Data Architecture

### Current Datasets

| Dataset / Source | Type | Usage |
| --- | --- | --- |
| `BW_2023_monthly.json` | Static historical regional data | Botswana carbon analytics |
| `BW_2024_monthly.json` | Static historical regional data | Botswana carbon analytics |
| `ZA_2023_monthly.json` | Static historical regional data | South Africa carbon analytics |
| `ZA_2024_monthly.json` | Static historical regional data | South Africa carbon analytics |
| `B_E_D.json` | Static regional energy dataset | Emissions Tracker regional calculations |
| `mockPowamov.ts` in-memory state | Simulated backend dataset | Nodes, telemetry, maintenance, analytics |
| `demo-api.ts` frontend fallback | Demo fallback dataset | Frontend resilience when backend is absent |
| `localStorage` auth/scenarios | Browser persistence | Users, sessions, calculator scenarios, tracker/client state |

## Mock Data Usage

The current platform uses mock data in three different ways:

- backend mock service:
  - Express returns generated node, telemetry, maintenance, and analytics data from in-memory logic
- frontend fallback data:
  - pages can render demo objects/arrays even if the API is unavailable
- frontend simulation:
  - some modules generate live behavior in the browser without using the backend

This layered approach makes the app resilient for demos, but it also means “working UI” should not automatically be interpreted as “integrated system.”

## Real-Time Simulation Logic

The main real-time simulation behaviors today are:

- backend telemetry updates on an interval in `mockPowamov.ts`
- frontend live emissions updates on an interval in `tracker.tsx`
- frontend digital twin animation and derived metrics in `digital-twin.tsx`

These are simulations, not actual device-ingested streams.

### Shared Contract Layer

A strong architectural choice already exists in the codebase:

- `lib/api-spec/openapi.yaml` defines the API contract
- `lib/api-client-react` generates typed React Query hooks
- `lib/api-zod` provides request/response schema support

This is a good base for future integration because it separates:

- backend route contracts
- frontend API usage
- validation concerns

### Database Readiness

The repository includes a real database package using Drizzle and PostgreSQL in `lib/db`, but that database layer is not yet central to the current product runtime. It should be understood as future infrastructure readiness rather than the current source of truth for the frontend experience.

## 8. Technology Stack

| Category | Technology |
| --- | --- |
| Frontend framework | React `19.1.0` |
| Frontend build system | Vite `^7.3.0` |
| Language | TypeScript `5.9.x` |
| Routing | Wouter |
| Server state | TanStack React Query |
| Local state | Zustand |
| UI primitives | Radix UI |
| Styling | Tailwind CSS `4.1.x` |
| Animation | Framer Motion |
| Icons | Lucide React |
| Charts | Recharts |
| Backend framework | Express `5` |
| Validation | Zod |
| API contract/codegen | OpenAPI + Orval |
| Logging | Pino / pino-http |
| Backend build | esbuild |
| Database layer | Drizzle ORM + PostgreSQL scaffold |
| Workspace tooling | pnpm workspaces |

# 9. Current Development Stage

## Overall Assessment

The current system is best described as:

- a polished partner-demo platform

- a strong UI prototype
- a partially structured MVP
- not yet a production operational system

## MVP Readiness

The app is strong enough for:

- partner demos
- design validation
- architecture planning
- workflow prototyping
- investor or stakeholder walkthroughs

The app is not yet strong enough for:

- production telemetry operations
- audited carbon reporting
- live customer onboarding
- secure multi-user production deployment

## What Is Production-Ready or Close

The strongest parts of the current system are:

- frontend information architecture and navigation
- module framing and product storytelling
- typed API contract structure
- local development experience
- Netlify-compatible static frontend build flow
- regional analytics pages based on bundled datasets

## What Is Simulation Only or Mock-Backed

These areas are currently simulated, mock-backed, or concept-only:

- POWAMOV live telemetry
- Command Center operational data
- Predictive Maintenance forecasts and alerts
- Digital Twin behavior
- Emissions Tracker live feed
- Operations Hub data
- authentication and account management

### Main Gaps Before Production

- real authentication and authorization
- persistent backend data storage used by active runtime flows
- device ingestion and telemetry normalization
- true real-time transport
- alerting logic grounded in sensor behavior or maintenance models
- environment/config separation and deployment hardening for multi-service hosting
- auditability for emissions calculations and factor governance

## 10. Future Integration Readiness

### POWAMOV Integration Readiness

Assessment: `Medium`

Why:

- the UI already expects node, telemetry, maintenance, and analytics endpoints
- the route structure is stable enough to support real backend substitution
- the biggest missing work is connecting those routes to actual POWAMOV data sources and persistence

### IoT Device Ingestion Readiness

Assessment: `Low to Medium`

Why:

- there is a clean API shape and a backend service boundary
- there is not yet an ingestion pipeline, broker, storage model, or device identity layer in active use
- the current backend still behaves as a seeded in-memory simulator

### Real-Time Telemetry Readiness

Assessment: `Medium on the frontend`, `Low to Medium on the backend`

Why:

- the frontend already renders live-style telemetry effectively
- the backend contract is ready for real data sources
- the current implementation does not include WebSockets, SSE, stream processing, or durable time-series storage

# 11. Recommended Next Technical Steps

If the goal is to move from partner demo to operational MVP, the highest-value next steps are:

1. replace mock telemetry generation with a real ingestion adapter for POWAMOV node data
2. define a persistent storage model for nodes, telemetry history, maintenance events, and user data
3. move auth from localStorage to a real identity/session system
4. formalize emissions factor sources and version them for calculator auditability
5. decide whether Digital Twin should remain a demo simulator or evolve into an engineering-grade model
6. add an explicit integration layer for regional and carbon datasets instead of bundling static snapshots only